

Heart Disease Prediction MLOps Project

Final Submission Report

BITS Pilani WILP – M.Tech Artificial Intelligence & Machine Learning
AIMLCZG523 – Machine Learning Operations (MLOps)

End-to-end MLOps workflow for binary heart disease prediction

Item	Details
Repository	https://github.com/dwai86/mlops-heart-disease
Dataset	UCI Cleveland Heart Disease Dataset
Task	Binary classification
Models	Logistic Regression and Random Forest
MLOps tools	MLflow, GitHub Actions, Docker, Kubernetes, Prometheus
Serving	FastAPI

Executive Summary

This project implements an end-to-end machine learning operations workflow for predicting the presence of heart disease using the UCI Cleveland heart disease dataset. The solution covers data acquisition, exploratory analysis, leakage-safe preprocessing, model development, cross-validation, hyperparameter tuning, experiment tracking, model packaging, automated validation, API serving, containerization, Kubernetes deployment and monitoring.

The implementation compares Logistic Regression and Random Forest classifiers. A scikit-learn Pipeline contains median imputation and standard scaling so preprocessing is learned only from the appropriate training data. GridSearchCV with five-fold stratified cross-validation is used for tuning, with ROC-AUC as the primary selection criterion. MLflow records experiment parameters, CV metrics, test metrics and artifacts.

The final model is exposed through FastAPI, packaged using Docker and validated in a local Kubernetes environment. GitHub Actions performs linting, automated tests and model training. Prometheus-compatible metrics provide lightweight runtime observability.

1. Introduction and Problem Definition

1.1 Objective

The objective is to demonstrate a complete MLOps lifecycle rather than only training a machine-learning model. The project therefore emphasizes repeatability, automated quality checks, experiment tracking, packaging, deployment and observability.

The selected use case is binary heart disease prediction. Given clinical attributes, the service predicts whether a record belongs to the no-disease class or disease-present class. This is an academic ML demonstration and is not intended to provide clinical diagnosis or medical advice.

1.2 Problem Formulation

Item	Implementation
Learning task	Binary classification
Dataset	UCI Cleveland Heart Disease dataset
Records	303
Columns	14 in processed Cleveland file
Predictors	13 clinical features
Original diagnosis	0–4
Binary target	0 = no disease; 1 = disease present
Primary selection metric	Cross-validation ROC-AUC
Final evaluation	Held-out test set

1.3 End-to-End Workflow

1. Acquire and validate the original UCI Cleveland dataset.
2. Perform EDA, missing-value analysis and class-balance analysis.
3. Create a stratified training/test split.
4. Build leakage-safe preprocessing and model pipelines.
5. Tune Logistic Regression and Random Forest with five-fold CV.
6. Track experiments and artifacts with MLflow.
7. Persist the best model pipeline as a reusable artifact.
8. Serve predictions using FastAPI.
9. Validate code using GitHub Actions, Ruff and pytest.
10. Package the API with Docker and deploy locally with Kubernetes.
11. Expose runtime metrics through the Prometheus-compatible /metrics endpoint.

2. Dataset Acquisition, Cleaning and EDA

2.1 Dataset Acquisition

The project uses the original UCI Cleveland heart disease package. The raw source is retained under `data/raw/heart+disease/`, with `processed.cleveland.data` used for machine-learning input. Keeping the original source package preserves reproducibility and makes the source data explicit.

The processed Cleveland file contains 303 records and 14 columns: 13 predictors and one target. Missing values are represented by '?' in the source and are read as missing values by pandas.

2.2 Target Transformation

The original diagnosis field contains values from 0 through 4. For the binary classification requirement, the implementation maps 0 to the negative class and every value greater than 0 to the positive class.

2.3 EDA

The EDA notebook provides the assignment-required analysis: dataset inspection, missing-value counts, target class balance, histograms for representative numerical variables and a correlation heatmap.

- Dataset preview and shape inspection.
- Missing-value count by feature.
- Binary target class-balance calculation.
- Histograms for age, trestbps, chol, thalach and oldpeak.
- Correlation heatmap for numeric variables.

2.4 EDA Purpose

The EDA identifies data-quality issues before modelling and supports the choice of evaluation metrics. Missing values require explicit handling, while class balance is relevant when interpreting accuracy, precision, recall and ROC-AUC.

3. Feature Engineering and Preprocessing

3.1 Feature Set

Category	Features
Demographic	age, sex
Clinical / symptom	cp, trestbps, chol, fbs
Cardiac test	restecg, thalach, exang
ST / exercise	oldpeak, slope
Diagnostic	ca, thal

3.2 Preprocessing Pipeline

The preprocessing sequence is SimpleImputer(strategy='median') followed by StandardScaler and the selected classifier. Both transformations are embedded in the same scikit-learn Pipeline as the estimator.

3.3 Leakage Prevention

The train/test split is performed before model fitting. During cross-validation, each fold fits imputation and scaling only on its training portion. Validation data is transformed using those learned parameters. This prevents information from the validation or test data from influencing preprocessing.

3.4 Reproducibility

- Fixed random state is used for the train/test split.
- StratifiedKFold uses five folds, shuffling and a fixed random state.
- The complete fitted preprocessing/model pipeline is serialized with joblib.
- requirements.txt and pyproject.toml support repeatable environment setup.

4. Model Development and Evaluation

4.1 Models

Logistic Regression is used as an interpretable linear baseline. Random Forest is used as a nonlinear ensemble capable of representing feature interactions. The two model families provide a compact but meaningful comparison for tabular clinical data.

4.2 Hyperparameter Tuning

Model	Search space
Logistic Regression	C: 0.1, 1.0, 10.0; solver: liblinear, lbfgs
Random Forest	n_estimators: 100, 200, 300
Random Forest	max_depth: None, 5, 10
Random Forest	min_samples_leaf: 1, 2, 4

4.3 Cross-Validation

GridSearchCV uses a StratifiedKFold strategy with five folds. Multiple scoring metrics are calculated: accuracy, precision, recall, F1 and ROC-AUC. The search refits the best configuration according to ROC-AUC.

4.4 Final Model Selection

After each model family is tuned, its best cross-validation ROC-AUC is compared with the other model family. The model with the highest CV ROC-AUC is retained as the overall best model. The held-out test set is then used for final evaluation rather than for tuning.

4.5 Metrics

Metric	Meaning
Accuracy	Overall proportion of correct classifications.
Precision	Correctness of positive predictions.
Recall	Ability to identify actual positive cases.
F1-score	Harmonic mean of precision and recall.
ROC-AUC	Discrimination/ranking ability across thresholds.

5. Experiment Tracking with MLflow

5.1 Tracking Strategy

MLflow is configured with the heart-disease-prediction experiment. Each model search is tracked as a run, including the selected estimator and best GridSearchCV parameters.

- Model name and selected hyperparameters.
- Cross-validation mean and standard deviation for accuracy, precision, recall, F1 and ROC-AUC.
- Held-out test accuracy, precision, recall, F1 and ROC-AUC.
- Confusion-matrix and probability-distribution plots.
- Serialized model artifacts.
- A final best-model run identifying the selected model and CV selection criterion.

5.2 Reproducibility and Packaging

The final pipeline is stored as `best_model.joblib`. Because preprocessing and the estimator are packaged together, the inference service can accept raw feature values without reconstructing the training transformations separately.

The repository also contains `requirements.txt` and `pyproject.toml`. The same training entry point is invoked locally and by the CI workflow, supporting reproducibility across environments.

6. FastAPI Model Serving

6.1 Service Endpoints

Endpoint	Purpose
GET /health	Health/readiness check
POST /predict	JSON inference request and prediction response
GET /metrics	Prometheus-compatible runtime metrics

6.2 Prediction Contract

The /predict endpoint accepts the 13 model features in JSON format and returns a prediction, the disease probability and a risk label.

Output	Validated example
prediction	0
probability	0.4445743342011674
risk_label	Low risk

The example above is the observed deployment-validation response documented for the project. It demonstrates that the service accepts the request and produces the expected response structure. It is not a clinical risk assessment.

6.3 API Observability

The API includes request logging and Prometheus-compatible metrics. The metrics endpoint allows monitoring of request activity and latency-related measurements.

7. CI/CD and Automated Testing

7.1 GitHub Actions

Stage	Implementation
Checkout	actions/checkout@v4
Python	actions/setup-python@v5, Python 3.11
Dependencies	pip install -r requirements.txt
Linting	ruff check .
Tests	pytest -q
Training	python scripts/train_model.py --data ... --artifact-dir artifacts

The workflow runs for pushes to the main/master branches and pull requests. A failed lint, test or training command returns a non-zero exit code and causes the validation job to fail.

7.2 API Tests

- GET /health must return HTTP 200 and status=ok.
- GET /metrics must return HTTP 200 and expose the expected application metric.
- POST /predict must return HTTP 200 and contain prediction, probability and risk_label.

7.3 MLOps Value

The CI workflow connects software-quality checks with the ML lifecycle. A change is validated not only for syntax and tests but also for the ability of the training entry point to execute successfully.

8. Containerization and Kubernetes Deployment

8.1 Docker

The Dockerfile packages the FastAPI service into a consistent runtime image. Deployment evidence records a successful image build and a running heart-disease-api container.

Validation	Observed result
Docker build	Successful
Container	Running
/health	{"status": "ok"}
/predict	Valid prediction response
Prediction	0
Probability	0.4445743342011674
Risk label	Low risk

8.2 Kubernetes

The Kubernetes manifest defines the heart-disease-api deployment and service. The deployment was validated using a local kind cluster named heartdisease.

Item	Observed result
Cluster	kind-heartdisease
Deployment	heart-disease-api
Pods	2 pods, both 1/1 Running
Service	heart-disease-api
Service type	LoadBalancer
Endpoint validation	kubectl port-forward

8.3 Deployment Interpretation

Because kind is a local cluster, a cloud LoadBalancer address is not expected. The API was therefore validated through kubectl port-forward. This is correctly treated as local Kubernetes validation rather than public-cloud deployment.

9. Monitoring and Observability

9.1 Prometheus Metrics

The API exposes a /metrics endpoint suitable for Prometheus scraping. The project includes monitoring configuration for collecting application metrics such as request activity and latency.

9.2 Logging

API middleware records request timestamps, paths and response status codes. This provides basic operational visibility and supports troubleshooting.

9.3 Scope

The monitoring implementation is intentionally lightweight to match the assignment. A production system could add dashboards, alerts, persistent metric storage, model drift detection, data-quality monitoring and security monitoring.

10. Architecture, Assignment Coverage and Conclusion

10.1 Architecture

The architecture separates data preparation, model development, serving, client access, containerization, orchestration, monitoring and CI/CD. The diagram below summarizes the complete lifecycle.

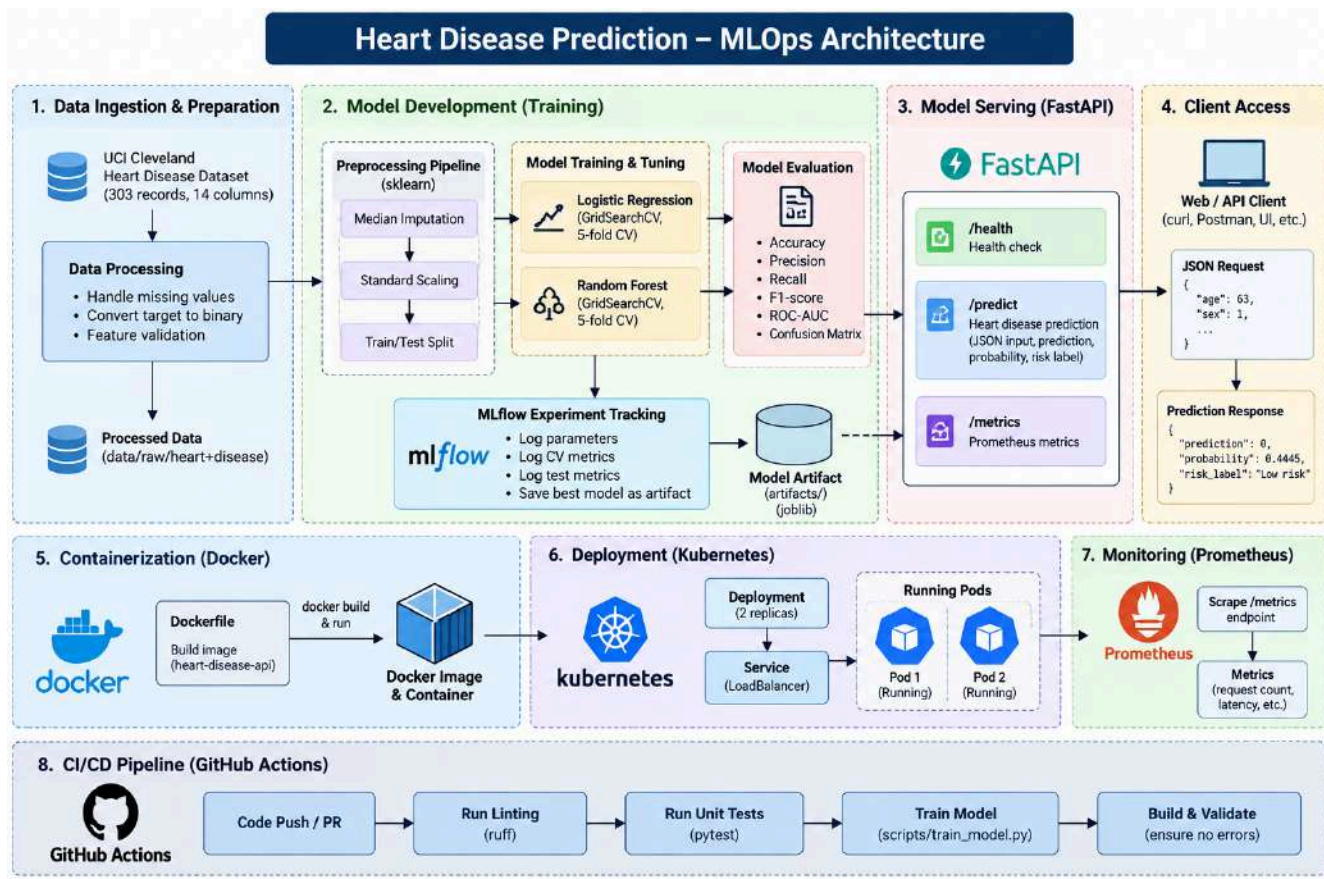


Figure 1. Heart Disease Prediction – MLOps Architecture.

10.2 Assignment Coverage

Assignment requirement	Project evidence
Data acquisition & EDA	Original UCI data, EDA notebook, missing values, class balance, histograms, heatmap
Feature engineering & model development	Leakage-safe pipeline, Logistic Regression, Random Forest
Experiment tracking	MLflow parameters, CV/test metrics, plots, model artifacts
Packaging & reproducibility	joblib, requirements.txt, pyproject.toml
CI/CD & testing	GitHub Actions, Ruff, pytest, training
Containerization	Dockerfile and validated Docker container
Deployment	Kubernetes manifest and local kind validation
Monitoring	Prometheus-compatible /metrics and request logging
Documentation	README, final report, architecture diagram, deployment evidence

10.3 Limitations and Future Improvements

- The project is an academic demonstration and is not a clinical diagnostic system.
- The dataset is small and represents the Cleveland subset rather than a broad modern clinical population.
- Monitoring is lightweight and could be extended with dashboards and alerting.
- Future versions could add model/data drift monitoring and automated model promotion.
- A production healthcare deployment would require stronger privacy, security, validation and regulatory controls.

10.4 Conclusion

The project demonstrates the complete MLOps lifecycle required by the assignment: UCI data acquisition, EDA, leakage-safe preprocessing, model development, cross-validation, hyperparameter tuning, evaluation, MLflow tracking, reusable packaging, automated testing, CI validation, Docker containerization, Kubernetes deployment and monitoring.

The combination of source code, automated validation, deployment evidence and the architecture diagram provides a reproducible path from raw data to a validated prediction API. The implementation remains intentionally focused on the assignment scope without adding unnecessary MLOps infrastructure.

Repository: <https://github.com/dwai86/mlops-heart-disease>

Key evidence files: `deployment_evidence.md`, `screenshots/architecture_diagram.png`, `notebooks/01_eda.ipynb`, `.github/workflows/ci.yml`